

Classes and Encapsulation

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is a Class?

- A **blueprint** for creating objects
- Describes an object's **data** (properties) and **behavior** (methods)
- Great for many similar objects: users, products, characters

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  sayHello() {  
    console.log("Hi, my name is " + this.name);  
  }  
}
```

Constructor, `this`, and `new`

- `new` creates an **instance** and runs the constructor
- `this` refers to the specific object being built
- Each instance gets its own properties

```
const alice = new Person("Alice", 25);  
const bob = new Person("Bob", 30);  
  
alice.sayHello(); // "Hi, my name is Alice"  
bob.sayHello();   // "Hi, my name is Bob"
```

Methods Are Shared

```
const alice = new Person("Alice", 25);  
const bob = new Person("Bob", 30);  
  
console.log(alice.sayHello === bob.sayHello); // true
```

- Properties are **per instance**; methods live on the prototype
- One function object, however many instances you create
- That is why methods are cheap and fields are not

Class Fields

- Declare properties with default values, outside the constructor

```
class Player {  
  score = 0;    // field with default  
  lives = 3;  
  
  constructor(name) { this.name = name; }  
  addScore(points) { this.score += points; }  
}  
  
const p = new Player("Sam");  
p.addScore(10);  
console.log(p.score); // 10
```

#private Fields

- A `#` prefix makes a field genuinely private
- Unreachable from outside the class body; not a convention, a rule

```
class BankAccount {  
  #balance;  
  
  constructor(start) { this.#balance = start; }  
  get balance() { return this.#balance; }  
}  
  
const acct = new BankAccount(100);  
console.log(acct.balance);    // 100  
// console.log(acct.#balance); // SyntaxError  
console.log(Object.keys(acct)); // [] - invisible
```

Getters and Setters

- Look like properties from outside, but run code

```
class Person {  
  #first;  
  
  constructor(first) { this.#first = first; }  
  
  get first() { return this.#first; }  
  
  set first(value) {  
    if (typeof value !== "string" || value.trim() === "") {  
      throw new Error("Name must be a non-empty string");  
    }  
    this.#first = value;  
  }  
}
```

- No parentheses at the call site: `p.first = "Ada"`

Validation Belongs in the Setter

```
const p = new Person("John");  
  
p.first = "Jane"; // fine  
p.first = "";    // Error: Name must be a non-empty string
```

- Because `#first` is private, **nothing can bypass the setter**
- A plain public property has no such guarantee
- This is the whole point of encapsulation: one place to enforce a rule

Computed Getters

- A getter can derive a value instead of storing one

```
class Employee {  
  #first; #last; #salary;  
  
  constructor(f, l, s) { this.#first = f; this.#last = l; this.#salary = s; }  
  
  get fullName() { return `${this.#first} ${this.#last}`; }  
  get salaryFormatted() { return "$" + this.#salary.toFixed(2); }  
}  
  
const e = new Employee("Jane", "Doe", 82500);  
console.log(e.fullName);           // "Jane Doe"  
console.log(e.salaryFormatted);    // "$82500.00"
```

Where Each Part Lives

Class Anatomy

Where each part actually lives: fields on the instance, methods on the prototype, statics on the class itself.

```
class Animal {  
  static kingdom = 'Animalia';  
  #secret = 'hidden';  
  constructor(name) { this.name = name; }  
  speak() { return this.name + ' makes a sound'; }  
  get label() { return this.name.toUpperCase(); }  
}  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);  
    this.breed = breed;  
  }  
  speak() { return super.speak() + ' (a bark)'; }  
}
```

class is syntax over the prototype chain, not a new object model: the same lookup rules apply. Two things it does add: #private fields are enforced by the language (unlike a leading underscore), and the whole class body runs in strict mode.

On the INSTANCE

name, breed
#secret
One copy per object. Anything assigned to this in the constructor.

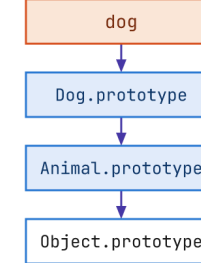
On the PROTOTYPE

speak(), get label
ONE copy, shared by every instance. This is why methods cost nothing per object.

On the CLASS

Animal.kingdom
Statics belong to the class, not to any instance. dog.kingdom is undefined.

What extends actually does



extends just links these. super(name) runs the parent constructor, and you must call it before touching this.

- Fields on the instance, methods on the prototype, statics on the class
- `class` is syntax over the prototype chain, not a new object model